Listen

Learning Doc 11: Stage 2, the Voice Agent
One section per issue, written as each issue closes

Implementation, stage 2 of 6

started 4 November 2026

Abstract

This document grows during stage 2: when an issue closes, its section is added here while the lessons are fresh, and the stage's last issue (S2-19) polishes the whole. Each section is taught from zero: the problem first, then the idea, an analogy and where it stops being true, the real code and real output, the usual mistakes, and a quick check (answers at the end). So far it covers ten issues: M1, M2, M3, M4, M5, M6, S2-08, S2-03, S2-04 and S2-01.

Contents

1 Before you start **3**

2 M1: pinning the tools that judge our code **3**
2.1 The problem . . . 3
2.2 The idea: pin . . . 3

3 M2: strict YAML, refusing duplicate keys **4**
3.1 The problem . . . 4
3.2 The idea: change one method of the parser . . . 4
3.3 Moved, not copied . . . 4
3.4 How the tests are written . . . 4

4 M3: an exit code with one meaning **5**
4.1 The problem . . . 5
4.2 The fix: 78 . . . 5
4.3 Why 78: the sysexits.h convention . . . 5
4.4 The guard test . . . 5

5 M4: a race test that can fail **6**
5.1 The problem . . . 6
5.2 Finding the real race first . . . 6
5.3 Making a rare race happen every time . . . 6
5.4 Proving the test works: break the code on purpose . . . 6

6 M6: a README that matches its folder **7**

7 S2-08: uv and the lock file **7**
7.1 The problem . . . 7
7.2 How the lock file is made . . . 7
7.3 What writes it, and what only reads it . . . 7

8 M5: testing Ctrl+C and a fatal stop **8**
8.1 The problem . . . 8
8.2 The idea: press Ctrl+C from inside the test . . . 8
8.3 What the review found: a broken stop path stopped pytest itself . . . 8

9 S2-03: the app index **9**
9.1 The problem . . . 9
9.2 The idea . . . 9
9.3 Close enough: difflib similarity . . . 9

10 S2-04: the actions Listen can take **10**
10.1 The problem . . . 10
10.2 The idea: a registry of allowed actions . . . 10
10.3 ctypes: calling Windows directly . . . 10
10.4 What the review found . . . 10

11 S2-01: command files and the recipe schema **11**
11.1 The problem . . . 11
11.2 JSON Schema: the rulebook is itself data . . . 11
11.3 A typo, traced . . . 12
11.4 Line numbers: LinedDict . . . 12
11.5 Skip the bad, keep the good . . . 13
11.6 Hot reload: a snapshot, and one assignment . . . 14
11.7 `listen commands check` . . . 14
11.8 What the two reviews found . . . 14

12 Key takeaways so far **16**

13 Answers **16**

1 Before you start

Doc 10
Everything from stage 1: the package and its command line, settings, logging, events, the bus, workers, the supervisor, locks, fakes and pytest.

An issue and a PR
An issue describes one piece of work (its file in `docs/issues/`); a pull request is the change that does it. `Closes #12` in a PR closes issue 12 when the PR is merged.

A dependency
A library our code needs. `pyproject.toml` lists the ones we ask for; they may need further libraries of their own.

A version number
`0.17.3` is MAJOR.MINOR.PATCH: patches fix bugs, minor versions add features, major versions may break things.

2 M1: pinning the tools that judge our code

2.1 The problem

`pyproject.toml` said `dev = ["pytest>=8.0", "ruff>=0.6"]`: "this version *or anything newer*". Every install, including each fresh CI machine, fetched the newest release that day. A new ruff release can add a check; CI then turns red on a commit that only touched the README, with no code change at all.

2.2 The idea: pin

Pinning fixes one exact version:

```
dev = ["pytest==9.2.0", "ruff==0.17.3"]
```

Written	pip / uv may install
`ruff>=0.6`	anything from 0.6 up, so the newest
`ruff==0.17.3`	exactly 0.17.3 (pinned)
`ruff~=0.17.3`	0.17.3, 0.17.4 … but not 0.18 ("compatible")

Only the tools that *judge* the code were pinned: if they change, CI's verdict changes. Libraries the code merely uses are pinned later by the lock file (Section 7).

Updated before starting. The issue file was written with 0.17.1 and 9.1.1. On the day M1 started, 0.17.3 and 9.2.0 were out, and unpinned CI was already using them, which is exactly the problem. The rule "update the issue file before starting" caught it.

Analogy

Agreeing with a teacher which *edition* of the textbook the exam uses. *Where it breaks:* editions change rarely; tools release every few weeks, so a pin must be raised on purpose from time to time (Dependabot now proposes those upgrades as pull requests).

Quick check 1 (answer in Section 13)

With `ruff==0.17.3` pinned and ruff 0.18.0 released tomorrow, which version does the next CI run install, and what must you do to use 0.18.0?

3 M2: strict YAML, refusing duplicate keys

3.1 The problem

PyYAML's `safe_load` silently keeps the *last* copy of a repeated key. Writing `thresholds:` twice in `settings.yaml` threw away the first block, and its values quietly returned to defaults. The new test showed the bug before any fix existed:

```
E       Failed: DID NOT RAISE SettingsError
```

3.2 The idea: change one method of the parser

A YAML *loader* builds Python dicts from the file. A small subclass overrides the one method that builds a mapping, and refuses a key it has already seen:

```
class _StrictLoader(yaml.SafeLoader):
    def construct_mapping(self, node, deep=False):
        seen = set()
        for key_node, _ in node.value:
            key = self.construct_object(key_node, deep=deep)
            if key in seen:
                raise _DuplicateKey(key, key_node.start_mark.line + 1)  # lines from 1
            seen.add(key)
        return super().construct_mapping(node, deep=deep)
```

Real result: `Can't start: settings.yaml:4: duplicate key 'thresholds'`, exit code 78 (after M3). The line is the *second* occurrence: the one to delete.

3.3 Moved, not copied

`settings.py` lost its own reading and parsing code (8 lines removed) and now calls the shared `load_yaml` (3 lines). Reading a **diff**: lines with `-` were removed, lines with `+` added, unmarked lines are context. The old messages for unreadable files and broken YAML stayed word for word, which is why all 71 old tests passed without edits.

Why move it now rather than later? Fixing it inside `settings.py` would have been the smaller change; a well-known rule says build for reuse only once a second user exists. Here the second user, S2-01's command files, was planned and two steps away, and the check is subtle code that should not exist in two copies. Both choices were defensible; the lesson is to say the trade-off in the PR, which is now a review rule in `CODING_STANDARDS.md`.

3.4 How the tests are written

A test file `test_*.py` holds functions `test_*`; each calls the code and `asserts` the result. `with pytest.raises(Error, match="..."):` passes only if that error, with that message, is raised inside the block. A fixture such as `tmp_path` is requested by naming it as a parameter; each test gets a fresh one. `@pytest.mark.parametrize` runs one test once per row of a table.

Watch out

Inside `pytest.raises(ValueError)` an expected `ValueError` makes the test *pass*: it is the outcome you asked for. Without `match`, *any* `ValueError` passes, even one raised for a different reason.

Quick check 2 (answer in Section 13)

After M2, someone writes `mode:` twice in `settings.yaml`. What appears in the terminal, and with which exit code does `listen run` end?

4 M3: an exit code with one meaning

4.1 The problem

When a program ends it hands back one number. The launcher (S2-17) will only see that number. Code **2** meant two things: argparse uses it for a wrong command (`listen bogus`), and we used it for a bad `settings.yaml`. The launcher could not tell "you typed a bad command" from "fix your settings".

4.2 The fix: 78

```
EXIT_BAD_SETTINGS = 78  # EX_CONFIG: the owner must fix settings.yaml; restarting would
    not help
```

Everything else already used the *name*, so one line changed. Real runs: `listen bogus` exits 2, a bad `settings.yaml` exits 78.

4.3 Why 78: the sysexits.h convention

`sysexits.h` is a list of exit codes from BSD Unix that many command-line programs follow. It starts at 64 to stay clear of the small numbers programs use freely.

Code	Name	Meaning
64	`EX_USAGE`	the command was used wrongly
65	`EX_DATAERR`	the input data was wrong
66	`EX_NOINPUT`	an input file is missing or unreadable
69	`EX_UNAVAILABLE`	a needed service is unavailable
70	`EX_SOFTWARE`	an internal error (a bug)
71	`EX_OSERR`	an operating-system error
74	`EX_IOERR`	an input/output error
75	`EX_TEMPFAIL`	temporary failure: try again later
77	`EX_NOPERM`	permission denied
78	`EX_CONFIG`	configuration error

(Also 67 no such user, 68 no such host, 72 a system file missing, 73 cannot create a file, 76 protocol error.) Python offers these names as `os.EX_CONFIG` on Linux and Mac, but not on Windows: only `os.EX_OK` exists there, so `app.py` writes 78 itself.

Why it is done this way

75 is the one code where restarting makes sense; 78 is one where it never does. Choosing codes with a known meaning lets the launcher's rules follow from the number alone.

4.4 The guard test

```
def test_unknown_command_exits_2(capsys):
    with pytest.raises(SystemExit) as exit_info:
        main(["bogus"])
    assert exit_info.value.code == 2
```

```
    assert "invalid choice: 'bogus'" in capsys.readouterr().err
```

- `"bogus"` is a deliberately fake command. argparse ends the program by *raising* `SystemExit(2)`, so the test catches it.
- `exit_info.value` is the caught exception; `.code` its exit code.
- `capsys` records printed text; `.err` is the error stream (stderr), where argparse writes; `in` checks the phrase appears somewhere in it.

It passed from the start: it is a *guard* that pins argparse's 2, so the "bad settings must not be 2" test stays meaningful.

Quick check 3 (answer in Section 13)

Windows denies Listen the microphone, and that stops the agent. Which code fits, and why not 78?

5 M4: a race test that can fail

5.1 The problem

The stage 1 review found that the bus's concurrency test passed even with the lock removed. A test that cannot fail protects nothing.

5.2 Finding the real race first

An experiment before writing the test: 8 threads subscribe 8 inboxes to a *brand-new* event type at the same moment, 20 trials each way:

```
lock=True:  inboxes that got the event, per trial (of 8): [8]
lock=False: inboxes that got the event, per trial (of 8): [1]
```

Without the lock, each thread sees "no list for this type yet", makes its own new list, and each overwrites the last: 7 subscriptions vanish. This is a **check-then-act** race: between checking and acting, another thread changes the answer. The old test never created a new list, so it could not see it.

5.3 Making a rare race happen every time

Creating a list normally takes far too little time for threads to collide reliably. The test slows that one moment down on purpose and starts all threads together:

```
def slow_new_list():
    time.sleep(0.01)
    return []

bus._subscribers = defaultdict(slow_new_list)   # private, on purpose
start = threading.Barrier(8)                    # all 8 threads go at the same moment
```

5.4 Proving the test works: break the code on purpose

The lock was removed locally (never committed) and the test run: it failed with `assert [0, 0, 0, 0, 0, 0, ...] == [1, 1, 1, 1, 1, 1, ...]`. With the lock back: 20 of 20 runs passed. Breaking code on purpose to check that a test notices is called **mutation testing**; here it was done by hand.

Analogy

Testing a smoke alarm by holding a lit match under it rather than waiting for a real fire. *Where it breaks:* the match is not a real fire; the slowed-down factory is not the real timing. It proves the lock protects the window, not that the window is ever this wide in practice.

Quick check 4 (answer in Section 13)

Why does a test that touches a private attribute such as `_subscribers` need a comment, according to the project's coding standards?

6 M6: a README that matches its folder

A README is the first thing a reader opens in a folder; a stale one misleads. `agent/README` still called `pyproject.toml` "added when we build", and never mentioned `uv.lock`, `.python-version`, `settings.yaml` or `logs/`. The fix was one line per file. The habit it teaches: when a change adds a file to a folder, its README gets a line in the same PR.

7 S2-08: uv and the lock file

7.1 The problem

Stage 2 brings big libraries (faster-whisper, onnxruntime, numpy). With only `>=` ranges, a silent upgrade could change speed or behaviour between the laptop and CI.

7.2 How the lock file is made

`uv lock` reads the wishes in `pyproject.toml` (3 names), asks PyPI which versions exist, follows each library's own dependencies (pytest alone brought 5 more), and picks one version of each that satisfies every rule at once. That search is called **resolving**. The result, `uv.lock`, records 9 packages with exact versions, download URLs and a **hash** (a fingerprint of the exact file):

```
name = "pytest"
version = "9.2.0"
dependencies = [
    { name = "colorama", marker = "sys_platform == 'win32'" },
    { name = "iniconfig" }, { name = "packaging" }, { name = "pluggy" }, { name = "
    pygments" },
]
```

A universal lock. A test project with `uv add pandas` resolved 6 packages but installed 4: the lock lists numpy twice (one version for Python 3.11, another for 3.12 and later) and `tzdata` only for Windows. The lock covers every machine; an install takes the lines that match this one.

7.3 What writes it, and what only reads it

Command	With the lock
`uv lock`	resolves and writes it
`uv add` / `uv remove`	edits `pyproject.toml`, re-locks, installs
`uv sync`, `uv run`	re-lock first if it is missing or out of date
`uv sync --locked`	never writes it: fails if it is out of date (what CI runs)
`uvx tool`	runs a tool once without adding it to the project

`uv.lock` is committed (the recipe); `.venv` is not (the meal, rebuilt from the recipe). Real numbers: CI's agent job went from about 30–36 s to 22–28 s.

Watch out

The first CI run failed: `Unable to resolve action astral-sh/setup-uv@v10`. Many actions publish a moving major tag (`actions/checkout@v4`); setup-uv only publishes exact tags such as `v10.3.3`. Pinning the exact tag fixed it. Dependabot now proposes real, existing newer versions of actions, so nobody has to guess tags.

Quick check 5 (answer in Section 13)

Someone adds a library to `pyproject.toml` by hand and forgets `uv lock`. What does plain `uv sync` do in CI, what does `uv sync --locked` do, and why is the second safer?

8 M5: testing Ctrl+C and a fatal stop

8.1 The problem

`run_agent` can end in two ways besides bad settings: the owner presses Ctrl+C (exit 0), or the supervisor gives up on a module (exit 3). Both had only ever been tried by hand. Shutting down is where bugs hide: a thread left running keeps the process alive, or a log line goes missing.

8.2 The idea: press Ctrl+C from inside the test

When you press Ctrl+C, Python raises `KeyboardInterrupt` *in the main thread*. The function `_thread.interrupt_main()` does exactly that from any thread. A `threading.Timer` calls it 0.2 s after Listen prints that it is ready:

```
def out(line):
    lines.append(line)
    if line.startswith("Listen is ready"):
        timers.append(threading.Timer(0.2, _thread.interrupt_main))
        timers[-1].start()
```

The test then checks the exit code (0), the printed `Stopping...`, the log line `stopped (exit code 0)`, and that no thread outlived `run_agent`. The fatal test does the same with `App.fatal("shell stopped responding")` and expects 3. It calls `fatal` directly: the supervisor only checks every 5 s, far too slow for a test.

Analogy

A fire drill: someone pulls the alarm on purpose, and you check that everyone leaves and the doors close. *Where it breaks:* a real Ctrl+C travels from the console through Windows' signal handling; `interrupt_main` skips that first part and starts at the `KeyboardInterrupt`.

8.3 What the review found: a broken stop path stopped pytest itself

The owner broke the code on purpose (the wait loop returning at once). Instead of a red `FAILED`, the output was `!!! KeyboardInterrupt !!!` and the rest of the suite was skipped: the timer's interrupt had hit pytest, which treats it as you pressing Ctrl+C. Two guards were needed, and the second was found only by trying the first:

When the interrupt arrives	Guard
after `run_agent` returned	`timer.cancel()` in a `finally`: it never fires
during the 1 s shutdown	`except KeyboardInterrupt: pytest.fail(...)`: a normal, named failure

"No thread left behind" became `assert set(threading.enumerate()) <= before`, comparing with the threads alive when the test started, so workers added in later stages are covered without editing the test.

Quick check 6 (answer in Section 13)

Why compare with the threads alive at the start instead of checking that no thread is named `shell` or `supervisor`?

9 S2-03: the app index

9.1 The problem

"Open Visual Studio Code" needs a list of the apps installed on this PC and where each one is. Windows keeps one already: every app in the Start Menu is a **shortcut** (a `.lnk` file) in two folders, one for you and one for every user.

9.2 The idea

Scan both folders for `.lnk` files (yours first, so your shortcut wins over the shared one with the same name), and read where each points with Windows' scripting object through **pywin32**. Measured: 125 shortcuts in 0.50 s, against 3.6 s through PowerShell. Skip uninstallers and links whose program is gone. The result maps a name to an `App(name, shortcut, target)`.

9.3 Close enough: difflib similarity

People mis-say names and speech recognition mis-hears them. `difflib.SequenceMatcher` scores two strings from 0 to 1: $ratio = 2M/T$, where M is the number of matching characters and T the two lengths added. Real values:

Heard	Best app	M	T	ratio	at 0.8
google chrome	Google Chrome	13	26	1.00	found
goggle chrome	Google Chrome	12	26	0.92	found
visual code	Visual Studio Code	11	29	0.76	refused
chrome	Google Chrome	6	19	0.63	refused
code	Visual Studio Code	4	22	0.36	refused

Below 0.8 the answer is "not found": opening the wrong app is worse than asking again. Short names like "chrome" get an **alias** instead (S2-01's command files).

Analogy

Your phone's contacts: "call Mom" looks up a name, and a small typo still finds her. *Where it breaks:* autocorrect always guesses something; the index refuses to guess below 0.8, because a wrong guess here launches or closes a real program.

Watch out

A test that cannot see the line. Every refused name in the first tests scored far below 0.8 (0.29 to 0.56), so with the threshold changed to 0.6 all 17 tests still passed. Boundary tests need values *just* under the line: "visual code" (0.76) and "chrome" (0.63) now fail at 0.6. PR #79 was merged before this fix, so it came as PR #81: the reason the pipeline now merges only after the review says "Ready".

A limitation found. "Notepad" is not found: on Windows 11 the built-in store apps have no `.lnk` in those folders.

Quick check 7 (answer in Section 13)

Work out the ratio for "crome" against "google chrome" ($M = 5$). Is it found, and what would make "crome" work without lowering the threshold?

10 S2-04: the actions Listen can take

10.1 The problem

Listen must *do* things (open a web page, launch, focus or close an app) without ever becoming "run any program": the words come from a microphone, and anyone in the room can talk.

10.2 The idea: a registry of allowed actions

Each action is one small file in `actions/primitives/`, registered by a **decorator** that records the function and its parameter types in a table:

```
@primitive("open_url", params={"url": str})
def open_url(system, url):
    if urlparse(url).scheme not in ("http", "https"):
        raise ActionError("open_url: only http and https addresses can be opened")
    ...
```

`run(name, system, **params)` refuses an unknown action, a missing, extra or wrong-typed parameter before anything happens. Apps are named, never given as a path: the name goes through the app index, so only installed Start Menu apps can be touched.

Analogy

A restaurant menu: the waiter takes orders only for dishes on it, and checks each order is complete. *Where it breaks:* a new dish is one reviewed file away; a menu at a restaurant changes far more slowly than code.

10.3 ctypes: calling Windows directly

Focusing and closing windows use Windows' own functions in `user32.dll`, through Python's standard `ctypes`, so no new library was needed. Every function's argument types are declared: otherwise a 64-bit window handle could be cut to 32 bits.

10.4 What the review found

- **Launch the shortcut, not its program.** Starting the bare target dropped the shortcut's arguments: Anaconda Prompt became a plain `cmd.exe`, Event Viewer (an `.msc` file) failed; 49 of 120 apps on the owner's laptop were affected. `ShellExecuteExW` starts the shortcut

the way a double-click does and still returns a process id. Checked for real: `cmd.exe /K ...activate.bat`.

- **Python's own errors quote their input.** `os.startfile` puts the whole address in its error message, and a traceback goes to the log. `open_url` now raises its own message `from None`.
- **Some windows are out of reach.** Event Viewer runs as administrator; a normal program may not close it (Windows calls this UIPI).

Watch out

`raise ActionError("...")` inside `except OSError:` still attaches the original error to the new one, and the traceback prints both, address included. `from None` leaves the original out.

Quick check 8 (answer in Section 13)

Someone says "open notepad.exe from C:\Windows". Which two parts of S2-04 stop Listen from running it?

11 S2-01: command files and the recipe schema

11.1 The problem

Listen needs a *menu*: the list of sentences you can say and what each one does. The owner asked the natural question: are these files hard-coded? The answer is no, and that is the design (spec 3.1, Doc 03: "commands are data, not code"). The files are written by hand, like `settings.yaml`, and opened in PyCharm. **YAML** is a plain-text format for nested lists and `key: value` pairs. Compare, first the hard-coded way (not what we do):

```
if text == "open youtube":
    open_url("https://youtube.com")
```

with the data way (what we do):

```
- id: youtube
  patterns: ["open youtube"]
  action: open_url
```

In the first, a new command means editing Python, running the tests and restarting Listen. In the second it is a few lines of YAML, picked up within 2 seconds, and later the AI builder and the web form can propose such entries because they are just data. The Python code only knows *how to read and check* commands, never *which* commands exist. The price of hand-written files is typos, so S2-01 is mostly about catching a typo and saying exactly where it is, without ever crashing Listen.

11.2 JSON Schema: the rulebook is itself data

A **schema** is a description of what valid data looks like. **JSON Schema** writes that description as data too (in `shared/recipe.schema.json`, draft 2020-12), and the `jsonschema` library checks any Python data against it. One rulebook means the agent, and later the server and the web form, all agree on what a valid command is.

Analogy

A paper form with rules printed on it: "tick exactly one of these two boxes", "this field takes only the words on this list". The clerk checks the form against the rules without knowing what the form is *for*. *Where it breaks:* the clerk checks the *shape* of a command, never whether it works: a well-formed `open_url` with a dead address passes. And the checker has

no idea which line of the file an error came from (see below).

These are the rules we used, each with the typo it catches:

Keyword	In our schema	Catches
`enum`	`action` is one of eight names (spec 3.5): the *allowlist*	`action: run_program`
`pattern`	a sentence's `{slot}` must be `query`, `target`, `app` or `number`	`open {nmae}`
`oneOf`	the entry has `action` *or* `steps`, exactly one	both, or neither
`if` / `then`	*if* the action is `press_keys`, *then* `when_app` is required	keys pressed in any window
`additionalProperties`	`false`: only the listed keys are allowed	`acton:`

The last one matters most: without it, `acton: open_url` would be silently ignored and the command would simply do nothing.

11.3 A typo, traced

The owner's example entry, with `action` misspelled on line 4 (the file starts with `commands:` on line 1):

```
commands:
  - id: search
    patterns: ["search {query}"]
    acton: open_url
    params: {url: "x"}
```

Step	What happens
1	YAML is parsed: valid YAML, so no error here.
2	The entry is checked against the `command` rule. jsonschema reports two things: `Additional properties are not allowed ('acton' was unexpected)`, and `{'id': 'search', 'patterns': [...], ...} is not valid under any of the given schemas`.
3	The second message is the `oneOf` failing (neither `action` nor `steps`) and it dumps the whole entry: unreadable. Our `_explain` function translates both into plain words.
4	The entry is skipped, the others still load, and one warning appears.

The warning, from the real code:

```
core.yaml:2: command 'search': unknown key 'acton'; needs either 'action' or 'steps'
```

To check *one entry* instead of the whole file, the schema keeps the `command` rule under `$defs` and the code points at it:

```
Draft202012Validator({"$defs": schema["$defs"], "$ref": "#/$defs/command"})
```

11.4 Line numbers: LinedDict

The warning above says `core.yaml:2`. Where did the 2 come from? jsonschema sees plain Python dicts, and a dict does not remember where in the file it was written. The YAML parser *does* know (it keeps a **mark** for each node, counted from 0, so we add 1 like editors do). The owner asked exactly how precise the line would be, so the real output of both tools was shown first, and then a choice was offered:

Option	Warning for the `acton` typo
A: the entry's line	`core.yaml:2: command 'search': unknown key 'acton' ...` (the line of `- id: search`; you find `acton` inside)
B: the exact key's line	`core.yaml:4: ... (did you mean 'action'?)`, about 15 more lines of code

The owner chose A (2026-11-08): simpler, and the entry is small enough to read. The loader in `core/yamlfile.py` (M2's file) now builds a `dict` subclass:

```
class LinedDict(dict):
    line = 0
    key_lines: dict = {}   # replaced per mapping by the loader

def construct_lined_map(self, node):
    data = LinedDict()
    data.line = node.start_mark.line + 1
    yield data
    data.update(self.construct_mapping(node))
```

Real values for the entry above: `entry.line == 2` and `entry.key_lines == {'id': 2, 'patterns': 3, 'acton': 4, 'params': 5}`. And because it is a `dict` underneath, it *equals* a plain dict, so all of M2's settings code and tests were untouched (135 tests passed, none edited). Why change M2's file here? S2-01 is the first caller that needs a line for every entry, not only for duplicates. Rejected: parsing the file a second time, because two parses of one file can disagree.

The review fills a gap. A target, default or alias is a single `name: value` line, and a string has no `.line`. So a wrong *second* default reported the section's first line. The review added `key_lines` (about 3 lines in the loader): such an entry now warns on its own key's line. Still option A, since the line is the entry's. One limit remains, written in the PR: a command written as plain text (`- just text`) reports the line of its `commands:` heading.

11.5 Skip the bad, keep the good

Two different failures need two different answers:

What is wrong	What Listen does
one *entry* (unknown key, bad risk, ...)	skip that entry with `file:line`; every other entry still loads
a whole *file* (broken YAML, duplicate key, top level not a mapping)	keep the file's *last good* data, and warn `; using the last good version`
a *deleted* file	its commands disappear at the next check

The last two rows differ on purpose, and the owner's design explains why: deleting a file is deliberate, while a broken file is usually a half-saved one (editors save in two steps), and a typo should not make your voice commands vanish mid-sentence.

Real run, a folder with `a.yaml` (command `hello`) and `b.yaml` (`bye`):

Step	Commands loaded	Problems
start	hello, bye	none
check again, nothing changed	hello, bye	none (no rebuild; returns `False`)
delete `a.yaml`	bye	none
save `b.yaml` as `commands: [`	bye (last good)	`b.yaml: not valid YAML ...; using the last good version`
re-create `a.yaml` broken the same way	bye only	`a.yaml: not valid YAML ...` with no "last good": it was forgotten

11.6 Hot reload: a snapshot, and one assignment

Every 2 seconds the housekeeping worker (S2-16, not built yet) will call `reload_if_changed()`. Reading and parsing every file every 2 seconds would be wasteful, so the check first takes a **snapshot**: for each `.yaml` file, its name, its modification time in nanoseconds and its size, which Windows gives without opening the file.

```
snapshot[path.name] = (stat.st_mtime_ns, stat.st_size)
...
if snapshot == self._snapshot:
    return False
self._snapshot = snapshot
self.library = self._build()   # one assignment: old or new, whole
return True
```

An edit, an added file or a deleted file all change the snapshot, so all three rebuild. The new library is built completely on the side, *then* stored in one assignment.

Analogy

A menu board in a café: the staff keep reading the old board until the new one is fully written, then hang it up in one move. Nobody ever reads half of each. *Where it breaks:* the café has one reader at a time; here several threads may read `.library` while it is swapped. It works because replacing one attribute is a single step in Python, so a reader holds either the whole old library or the whole new one.

Files load in **sorted name order**, which makes "the earlier entry wins" (a tie, or a repeated `id`) the same on every run and every PC. Otherwise it would depend on the order Windows lists the folder. The owner's rename example: `media.yaml` → `music.yaml` keeps its place (after `core`, before `routines`); renaming it `aaa.yaml` would move its commands to the front, and they would start winning ties.

11.7 `listen commands check`

The same loader, run once by hand. It prints each problem and a summary, and exits 1 if there was any problem (not 2, which argparse uses for a wrong command, nor 3 or 78 from M3). On the starter files:

```
14 commands, 5 targets in 5 files: no problems          (exit 0)
```

The owner broke `core.yaml` by hand to see it fail (`launch_app` became `launch_apps` on line 14), then restored it with git:

```
core.yaml:12: command 'open': action: 'launch_apps' is not an allowed action
13 commands, 5 targets in 5 files: 1 problem             (exit 1)
```

Line 12 is where the `open` entry starts, not the typo's line 14: option A. The other 13 commands loaded. If the `commands/` folder itself is missing (a moved checkout), the check prints `<folder>: no such folder` and exits 1; the first version said "no problems", which would have been false comfort. Listen itself still starts, with an empty menu.

11.8 What the two reviews found

A linter trap (B023). The first draft of the loader defined small helper functions *inside* the loop over files, and the helpers used the loop's variable `name`. Ruff refused: `B023 Function definition does not bind loop variable 'name'`. A function remembers the *variable*, not the value it had when the function was written, so a helper called after the loop had moved on would see the last file's name. Rather than silence the rule, the loader was split into `_build` (the loop) and `_merge`,

which receives the file name as an argument. Because the code changed, the by-hand breakages were run again on the new version (five caught).

Watch out

A passing test is not yet a guard. After the first review, two new tests passed immediately on working code, which proves nothing about the line each one protects. Each guard was removed on purpose and the test run:

Guard removed	Result
`skipped.update(error.path)` (a file with a bare `commands:`)	`TypeError: 'NoneType' object is not iterable`
`del self._last_good[gone]` (forget a deleted file)	`assert ['hello'] == []`

Before these tests existed, removing those two lines failed *no* test. This is M4's lesson again: break the code to see the test notice.

A test that passed for the wrong reason. The test "`listen commands check` passes on the starter files" was written before any starter file existed, and it passed at once: an empty `commands/` folder has no problems either (`0 files: no problems`). It was tightened to expect `targets in 5 files: no problems`, seen to fail, and passed only once the five files were added (transcript, 2026-11-08). A green test proves something only if it could have been red.

Confirmation follows risk (the owner's rule). The first review noticed that spec 3.6 makes closing an app high risk, but the code only wrote `risk: low`. A table of defaults per action was added. Then the owner asked for more: any command with `risk: high` should ask for a confirmation, "this way I can add more commands without changing the code". The code became:

```
ACTION_RISK = {"close_app": "high"}     # high risk when the file doesn't say

risk = entry.get("risk", ACTION_RISK.get(action, "low"))
continuous = entry.get("continuous", "confirm" if risk == "high" else "allow")
```

In the YAML	risk	continuous mode
`action: close_app`, nothing else	high	asks first
any command with `risk: high`	high	asks first
a routine (`steps`), nothing else	low	runs at once
`close_app` with `risk: low`	low	runs at once
`risk: high` and `continuous: allow`	high	runs at once (the file wins)

A new risky command is now one word of YAML. The rule is the same idea as the whole issue: move the decision out of code and into data. The second review also found that the spec (3.7) and the S2-07 issue still described the old "only `close_app` confirms" rule, so both were changed in the same PR: the spec is the source of truth, and a second place stating a safety default drifts.

Quick check 9 (answer in Section 13)

(a) While Listen runs, you delete `media.yaml`, and a minute later save a new `media.yaml` containing only `commands: [`. Are the media commands available after each step, and what does the second step report? (b) You add a command with `risk: high` and nothing about `continuous`. What happens in continuous mode, and which Python file changes?

12 Key takeaways so far

Issue	Mental model
M1	tools that judge the code get exact versions, raised on purpose
M2	refuse silently lost data; one shared loader, not two copies
M3	an exit code is a contract: one number, one meaning
M4	a test you never saw fail proves nothing; break the code to check it
M6	a folder's README changes in the same PR as the folder
S2-08	`pyproject.toml` says what is allowed; `uv.lock` says what was chosen
M5	a test that interrupts must make sure the interrupt cannot escape into pytest
S2-03	test a threshold with values just under it; refuse rather than guess wrong
S2-04	a menu of allowed actions; apps by name only; errors never quote what was said
S2-01	commands are data checked at the edge: skip the bad entry, keep the last good file; confirmation follows `risk`, set in YAML

13 Answers

1. Still 0.17.3: the pin allows nothing else. To use 0.18.0, change the pin, run `uv lock` and `uv sync --extra dev`, run ruff and pytest, fix anything new, and merge that as its own PR (or merge the PR Dependabot opens for it).

2. `load_yaml` raises `YamlFileError` (`settings.yaml:N: duplicate key 'mode'`); `load_settings` turns it into `SettingsError`; `run_agent` prints `Can't start: settings.yaml:N: duplicate key 'mode'` and returns 78.

3. 77, `EX_NOPERM` (permission denied). 78 would send you to fix `settings.yaml`, which is not the problem.

4. Reaching into internals ties the test to details the public interface hides; the comment tells the next reader why the public interface was not enough, so they do not "tidy" it away or copy the pattern without reason.

5. Plain `uv sync` quietly re-locks with whatever is newest that day, so CI tests versions nobody reviewed. `--locked` refuses and fails, so the mistake is caught and the lock is updated deliberately in the PR.

6. The names catch only the threads stage 1 has. Later stages add workers (voice, storage) with new names; comparing with the threads alive before the test covers them all, and ignores pytest's own threads.

7. $2 \times 5/(5 + 13) = 10/18 = 0.56$: refused. An alias `crome -> Google Chrome` in a command file would match it exactly (1.0); so would the speech model hearing it right.

8. There is no action that runs a path: `launch_app` takes an app *name*, and the name must be found in the app index (Start Menu shortcuts only). A path is not a name in that list, so it is refused with "no installed app matches that name".

9. (a) After the delete: no, the media commands are gone (a deliberate delete drops them, and the file's "last good" copy is forgotten too). After the broken save: still gone. The file has never parsed since it came back, so there is nothing to fall back on; the check reports `media.yaml: not valid YAML (...)` without the words "using the last good version". (b) The command asks first, because `risk: high` defaults to `continuous: confirm`; no Python file changes, only the YAML. (Writing `continuous: allow` in the YAML would override it.)